Ship Real AI

Ship Real AI

The AI Engineer Bootcamp for LLMs, RAG, MCP,
AI Agents and Evals, from First Principles
to Production and Interviews

Sridhar Mukkandi

Draft

Ship Real AI: The AI Engineer Bootcamp for LLMs, RAG, MCP, AI Agents and Evals, from First Principles to Production and Interviews

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

About the examples in this book. Tarkika, its customers and its staff are imagined. Every number about Tarkika, and every price marked as an example, is assumed, to explain the ideas clearly; none is a measurement of a real company or product. Facts about real companies, products and incidents are cited.

Products and services change quickly. Model names, versions and prices were checked on the date given in the version register at the back of the book. All trademarks belong to their respective owners.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or financial advice.

Text set in IBM Plex Serif, Work Sans and IBM Plex Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vi

I The job and the foundations vii

1 What an AI engineer builds 1

2 How LLMs work, for engineers 7

3 Prompts and context 18

Appendices 28

References 28

About the author 29

Preface

The preface is written last, once the modules are final.

PART I

The job and the foundations

These modules explain what a model can and cannot do, and everything an AI engineer builds around it.

01

What an AI engineer builds

A language model can write a polite reply in Hindi, explain a return policy, or sort a thousand complaints in minutes. Yet most of the work of putting one in front of real customers has little to do with the model itself. This module shows what a model can and cannot do, and what an AI engineer builds around it.

1.1 What does a language model actually do?

You send it text, and it sends back text. That is all a single call to a model does.

GPT, Gemini, Claude, Llama, Qwen and DeepSeek are names of models. It helps to keep three words apart. The **model** is the trained program that turns text into text. The **model provider** is the company that runs it and sells access, such as OpenAI, Google or Anthropic. The **application** is a product built on top, such as the ChatGPT app. ChatGPT is not a model; it is an application that uses OpenAI's models and adds a lot around them.

Now ask a few simple questions about what a model can do on its own.

Ask it about an order placed this morning. It cannot answer. A model learns from data collected up to a certain date, and knows nothing after that. It has also never seen your company's database. This limit is called the **knowledge cutoff**.

Ask a question, then a follow-up such as "and how long will that take?" It does not know what "that" means. Every call to a model is separate, and the model keeps nothing from one call to the next. Chat apps seem to remember because the app sends the earlier messages again with every new one. Module 4 builds this.

Paste in three hundred pages of policy documents and a year of emails. It will refuse, or fail. Each call accepts only a limited amount of text, called the **context window**.

Ask the same question twice. You can get two different answers. An ordinary program gives the same output for the same input, every time. A model picks its words by chance, so its output can change from one call to the next. This is the biggest difference between AI systems and ordinary software.

Ask it to check an order, send an email or issue a refund. It cannot. It can only write text. It can write "I have issued your refund", but writing that sentence does not move any money.

1.2 Three kinds of AI work

People who "work in AI" usually do one of three jobs. Some **make models**: they collect data and train models, which needs large research teams and large budgets. Some **serve models**: they make trained models run fast and cheaply on hardware, which is called inference engineering. And some **build applications**: they take a model that already exists and turn it into a product that works for real users.

This book is about the third job.

1.3 If the model cannot act, what can?

In software, what does the actual work, such as looking up an order or sending an email? Code. A function such as `check_order_status(order_id)` can query a database and return the answer.

Can that function understand a customer who writes "mera parcel kab aayega"? No. It needs an exact order number, passed in exactly the right way. The model understands the customer perfectly, but cannot query anything. Each one has exactly what the other lacks.

So put them together. The model reads the customer's message and decides which function should run and with which values. Your program runs the function. The result goes back to the model, which writes the reply (Figure 1.1). A model connected to functions it can ask your program to run is called an **agent**, and each such function is called a **tool**.

Now look at one particular kind of tool: one that fetches information, such as a search over the company's policy documents. The model asks for the relevant paragraph, the paragraph is added to the question, and the model answers using it. That is **retrieval-augmented generation**, or **RAG**. The name describes the three steps in order: *retrieve* the information, *augment* (add to) the question with it, then *generate* the answer. RAG is not a separate technology. It is an agent whose tool fetches facts. Modules 6 to 8 are about doing it well.

You will also see agents described in other ways. "Model plus tools" came first. Then people noticed that what you put in front of the model decides most of the result, and started saying "model plus context". The newest description is "model plus harness",

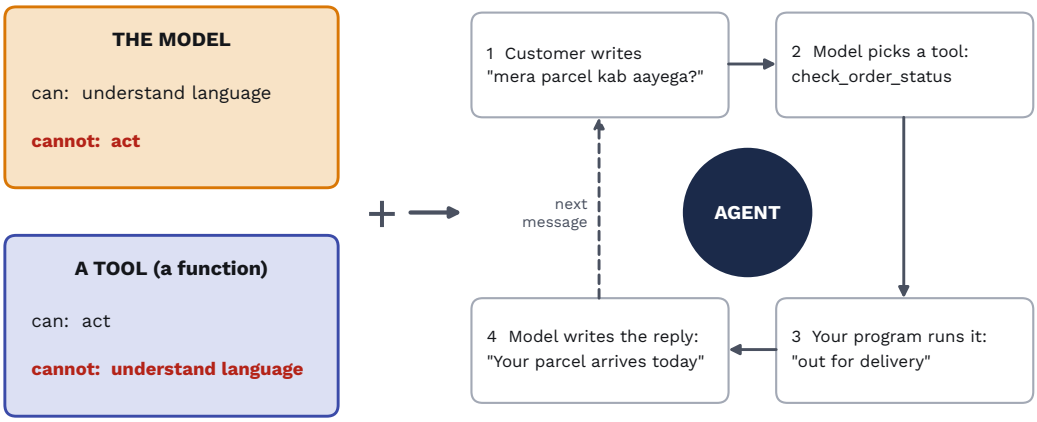


Figure 1.1 A model understands language but cannot act. A tool can act but cannot understand language. Connected, the model chooses the tool and your program runs it.

where the **harness** means everything around the model: the instructions, the tools, the memory, the checks, the code that runs the loop. All three descriptions are true. Building that harness is the job.

1.4 What does an AI engineer build around the model?

Figure 1.2 shows the parts, in the order a customer's message passes through them. Each part has a module of its own.

1. **The front door** receives the message, and limits how many requests come in (Module 16).
2. **Safety checks** look at what comes in: private data, abuse, hidden instructions (Module 12).
3. **Control** decides what happens next. A fixed sequence of steps, or an agent that chooses, or a plan for a long task (Modules 11 and 14).
4. **The model** is called, perhaps several times (Modules 2 and 10).
5. **Context** decides what the model sees on each call: the instructions, the retrieved documents, the memory of the conversation (Modules 3, 4 and 6 to 8).
6. **Tools** let it act, through functions you write or through standard connectors such as MCP (Modules 5 and 13).
7. **Safety checks** look at what goes out, before the customer sees it or a refund is issued (Module 12).
8. Under all of it, **quality and operations**: tests that show whether it works, traces of every run, cost and caching, safe releases (Modules 9, 15 and 16).

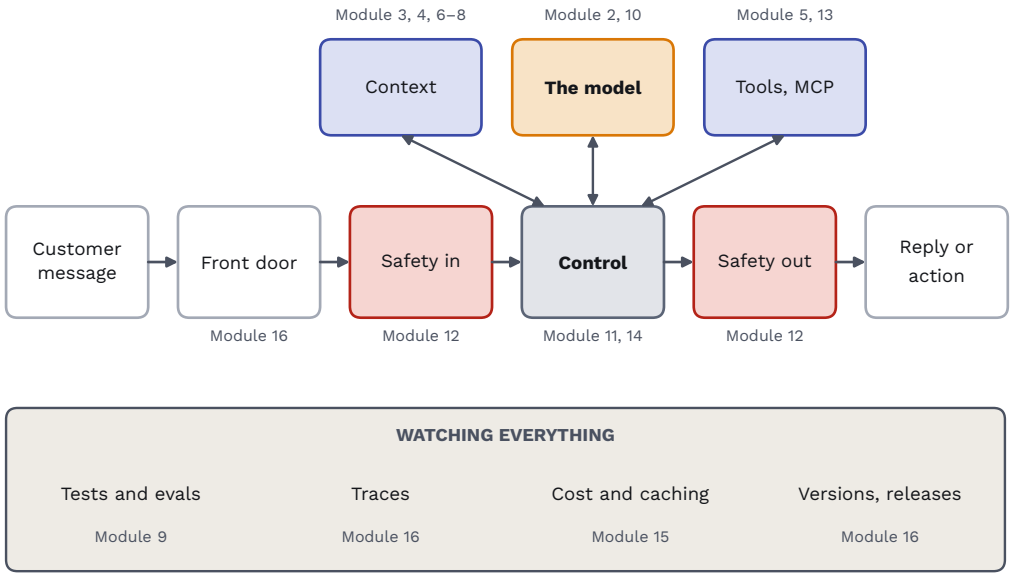


Figure 1.2 The parts of an AI system. A message enters at the front door, is checked, is handled by the control layer with the model, its context and its tools, and is checked again on the way out. Tests, traces, cost and caching watch over everything. The numbers are the modules that cover each part.

The model is one box out of eight. The other seven are what the engineer builds.

1.5 Calling a model is not calling a function

In code, a model call looks like any other function call: text goes in, text comes out. It behaves very differently.

	An ordinary function	A model call
Speed	Microseconds	Seconds
Failure	Rarely fails	Fails sometimes: busy servers, limits, timeouts
Same input	Same output, every time	A different output is possible every time
Cost	Free to run	Charged for every call, by the amount of text

Each row changes how you write code around the model. Because the output varies, you check every reply before your code uses it (Module 5), and you cannot test it the

usual way (Module 9). Because calls fail, you retry some failures and not others (Module 11). Because calls are slow, you run many at once, within the provider's limits (Module 15). And because every call costs money, you count what you send (Modules 3 and 15).

1.6 Should this problem use AI at all?

The table above gives a simple rule. A model is slower, costlier and less predictable than ordinary code. So when ordinary code can do the job, use ordinary code.

"Was this order placed more than 7 days ago?" is a date comparison. One line of code answers it correctly every time, for free. "Is this customer angry, and are they asking for a refund?", written in a mix of Hindi and English with spelling mistakes, cannot be written as a rule. That is where a model earns its cost.

Before building, ask four questions:

1. Is the input messy language, or a judgement that no rule can capture? If not, write the rule.
2. If the model is wrong, will someone notice, and how much will it cost?
3. What does success look like, as a number you can measure before and after?
4. Can you buy this ready-made? A generic support bot may be enough. You build your own when you need your own data, your own rules, or control over what it is allowed to do.

1.7 Your first call, and what it costs

Here is a complete call to a model in Python. It uses an OpenAI model through **LangChain**, a popular library that gives the same way of calling models from many providers. To switch to another provider, you change the import and one line.

```
from langchain_openai import ChatOpenAI

MODEL = "gpt-5-mini"             # model names change; keep this in one place
llm = ChatOpenAI(model=MODEL)    # reads OPENAI_API_KEY from the environment

reply = llm.invoke("Can I return opened earbuds after 10 days?")
print(reply.content)             # the answer
print(reply.usage_metadata)      # how much text went in and came out
```

The answer will be polite and sensible, and probably wrong for your shop, because the model has never seen your return policy. The last line prints the amount of text that went in and came out, counted in **tokens**, the small pieces of text that models read and charge by. Module 2 shows what they look like.

Now work out the cost. Suppose the provider charges ₹350 for every million input tokens and ₹1,700 for every million output tokens. These are assumed example prices; output usually costs several times more than input.

1. This question was about 30 tokens in and 120 tokens out: 30 × 350 ÷ 10,00,000 + 120 × 1,700 ÷ 10,00,000, about ₹0.21.
2. A real support request also carries instructions and the relevant part of the policy: say 3,000 tokens in and 300 out. That is 1.05 + 0.51, about ₹1.56 per message.
3. At 4,000 messages a day, that is about ₹6,240 a day, or about ₹1,87,000 a month.

Two things stand out. Most of the cost comes from the text sent *with* each question, not from the question itself. And the bill grows with every message, so a small saving per call becomes a large saving per month. Modules 3 and 15 come back to both.

1.8 The project for this book

Through the book, you will build one system: a customer-support agent for **Tarkika**, an imagined online marketplace that sells phones, electronics, fashion and home products across India. By the end, it will:

- answer questions from the company's policy documents, with the source of each answer, and say "I don't know" when the documents do not cover a question;
- look up orders, and prepare refunds that a person approves above a set amount;
- remember a customer's earlier messages, and keep each customer's data separate;
- refuse hidden instructions in customer emails;
- report its own quality, cost and speed.

Each of these is one part of Figure 1.2. First, though, the model in the middle: how it reads text, and why you pay for every piece of it. That is Module 2.

02

How LLMs work, for engineers

You do not need to build models to use them well. But a few facts about how they work explain most of what you will see in practice. This module shows them, mostly in pictures, and points you to free tools where you can try them yourself.

2.1 Tokens: what the model actually reads

A model does not read words the way we do. It reads small pieces of text called **tokens**. Figure 2.1 shows how a real tokenizer, the one used by OpenAI's GPT-4o models, splits a few words.

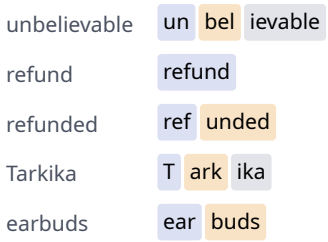


Figure 2.1 Real splits from the GPT-4o tokenizer. Common words are one token; longer or rarer words are cut into pieces.

"Refund" is common, so it is one token. "Refunded" is two. "Unbelievable" is three. A made-up name like "Tarkika" is cut into "T", "ark" and "ika", because the model has never seen it as a whole. In English, 100 tokens is roughly 75 words.

Why should you care? Because everything about a model is counted in tokens. The price is per token. The speed is per token. The limit on how much you can send is in tokens.

Do not confuse a token with a **chunk**. A token is a tiny piece the model reads. A chunk is a large piece of a document, often a few hundred tokens, that you cut out yourself for searching. Chunks come in Module 7.

Try it: search for “OpenAI tokenizer” or open tiktokenizer.vercel.app. Paste any sentence and watch it split into coloured pieces, exactly like the figures here.

2.2 The same message in five languages

Now look at Figure 2.2. It is one message, “My refund has not come yet”, written five ways, and split by the same tokenizer. A dot marks a space.



Figure 2.2 One message, five ways, split by the GPT-4o tokenizer. Each coloured box is one token. The dotted circles in Hindi, Telugu and Kannada are pieces of a letter that the tokenizer cut away from the rest.

English needs 6 tokens. For the same meaning, Telugu needs 13, Hindi 14 and Kannada 16. In these scripts, the tokenizer often cuts a single letter into two pieces, like the vowel signs standing alone in their own boxes.

Why? The tokenizer learned its list of pieces mostly from English text. Common English words got their own tokens. Words in Indian scripts were seen less often, so they are built from many small pieces.

Figure 2.3 shows that this is improving, but not gone. The older GPT-4 tokenizer needed 29 tokens for the Hindi message, 36 for Telugu and 46 for Kannada. Studies across many languages find gaps of three to five times for Hindi, and up to fifteen times between some languages (*Reducing Tokenization Premiums for Low-Resource Languages* 2026; Petrov et al. 2023).

For your work, more tokens for the same message means a higher bill, a slower reply, and less room for anything else in the request. Hinglish, written in English letters, sits in between. If your users write in Indian languages, count a sample of their real messages before you plan a budget.

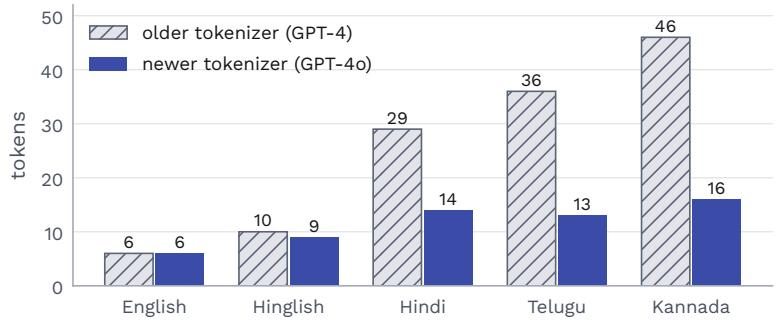


Figure 2.3 Tokens for the same message, counted with the older GPT-4 tokenizer and the newer GPT-4o tokenizer.

2.3 Watching the model write

A model writes one token at a time. At each step it looks at all the text so far, gives every possible next token a chance, picks one, and adds it. Then it repeats. Figure 2.4 shows four steps.

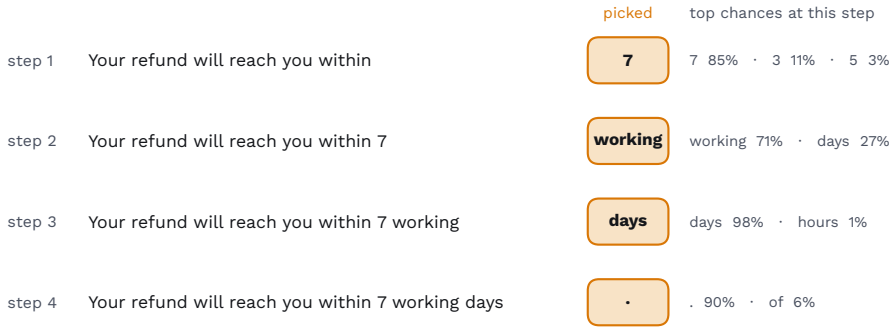


Figure 2.4 The model writing a reply, one token per step. At each step it picks one token from many possible ones. (Chances made up for illustration.)

At step 1, “7” had an 85% chance, so it was picked. But “3” had 11%. Run the same request again, and sometimes the reply will say 3 days, not 7. Nothing is broken. The model is picking by chance, and the chances can go either way.

A setting called **temperature** controls how strongly the model prefers its top choice (Figure 2.5). At a low temperature, the top token wins almost every time. At a high temperature, the chances spread out and replies vary more.

So your code must never expect one exact reply.

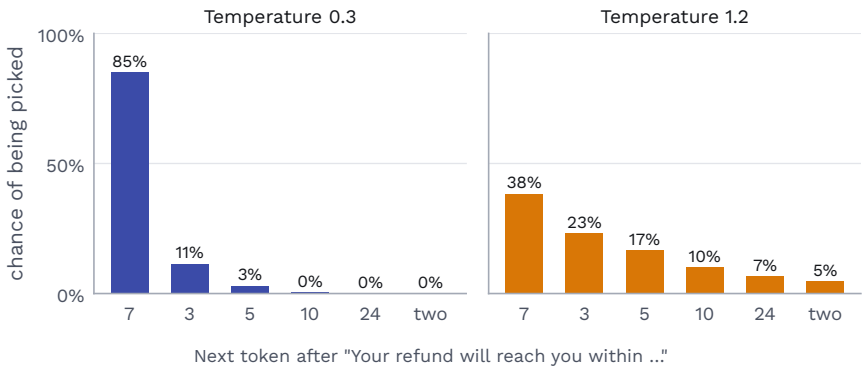


Figure 2.5 The same chances at two temperatures. Low temperature makes the top choice dominate; high temperature spreads the chances out. (Numbers made up for illustration.)

2.4 How a model learns

Training is one simple exercise, repeated an enormous number of times (Figure 2.6). Hide the next word of a sentence. Let the model guess it. Check the guess. Then adjust the model a little, so that the right word becomes slightly more likely next time.

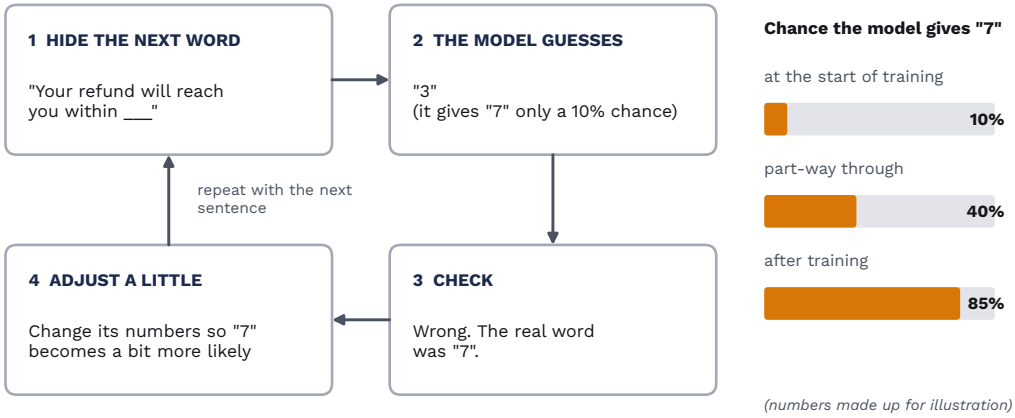


Figure 2.6 Training in four steps. On the right: the chance the model gives the right word grows slowly as the loop repeats. (Numbers made up for illustration.)

One round changes almost nothing. Trillions of rounds, over text from the whole web, change everything. To get good at guessing the next word, the model has to learn grammar, facts and common patterns.

What gets adjusted are the model's internal numbers, called **parameters**. A large model has billions of them, and this is the one fact about training that affects your work: every parameter must sit in memory (Figure 2.7).

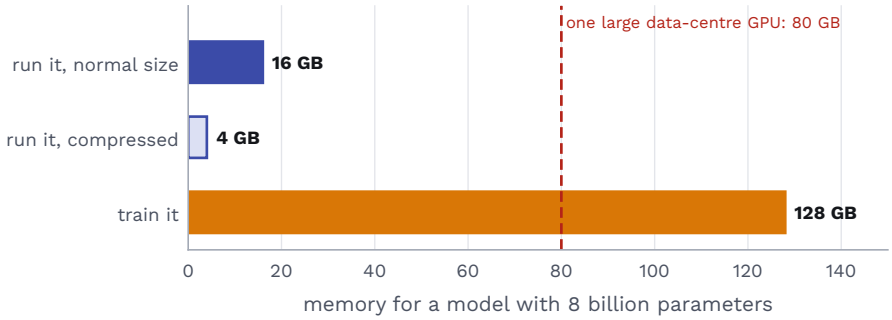


Figure 2.7 Memory needed for a model with 8 billion parameters. Running it fits on one large GPU. Training it does not.

Each parameter usually takes 2 bytes, so a model with 8 billion of them needs about 16 GB to run. Compressed to half a byte each, it needs about 4 GB. Training needs about 16 bytes per parameter, roughly 128 GB, more than one large GPU holds (Rajbhandari et al. 2020). That is why most companies run models that someone else trained.

2.5 How a word gets its meaning from its neighbours

"Bank" means the side of a river in "the river bank", and a place for money in "my bank account". The model works out which meaning applies the same way you do: by looking at the words around it. In Figure 2.8, "bank" pays most of its attention to "river".

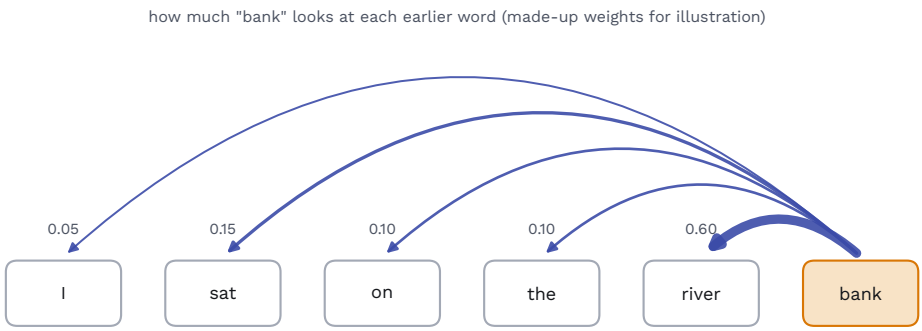


Figure 2.8 Each word looks at the other words to understand itself. Here "bank" looks mostly at "river". (Numbers made up for illustration.)

This idea is called **attention**. It is the heart of the **transformer**, the design almost every modern model uses.

Attention has a cost you will meet in practice. To write each new word, the model looks back at every word before it. To save time, it keeps notes on the earlier words in memory, and the notes grow with every word. For a typical 8-billion-parameter model:

1. One conversation of 8,000 tokens needs about 1 GB of notes.
2. Fifty such conversations at once need about 50 GB, three times the memory of the model itself.

That is why long conversations are expensive, and why every model limits how much text it accepts.

Try it: Transformer Explainer (poloclub.github.io/transformer-explainer) runs a small model in your browser. Type a sentence and watch the attention lines move between words, then see the chances for the next token (Cho et al. 2025).

2.6 Meaning as a map

"I want my money back" and "refund chahiye" share no words, yet mean the same thing. A search for matching words would never link them.

An **embedding** turns a piece of text into a long list of numbers, so that texts with similar meaning get similar lists. Think of each list as a point on a map. In Figure 2.9, refund messages land in one area, delivery complaints in another, and product questions in a third, whatever language they are written in.

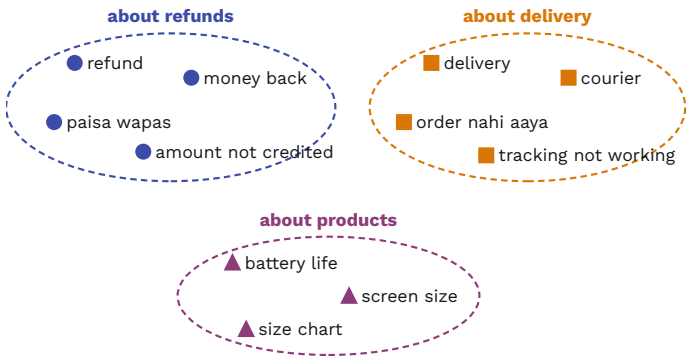


Figure 2.9 Messages placed by meaning. Real embeddings have hundreds of numbers; this picture squeezes them into two so they can be drawn. (Positions illustrative.)

To check whether two messages mean the same thing, you measure how close their points are. The usual measure is **cosine similarity**, which gives a score near 1 for very similar meanings.

How long should the list of numbers be? Longer lists capture finer differences in meaning, but take more space and more time to search. A million stored pieces of text take about 6 GB with 1,536 numbers each, and about 1.5 GB with 384. Pick the smallest size that still finds the right results on your own data.

Embeddings come from separate, smaller models that only produce these lists. Module 6 builds search on top of them.

Try it: the Embedding Projector (projector.tensorflow.org) shows thousands of words as points in 3D. Search for a word and see which words sit closest to it.

2.7 More text can mean worse answers

Every model has a **context window**: the most tokens it can take in one call. Some now hold a million tokens or more. So it is tempting to send everything: all the policies, all the past emails, every order.

Look at Figure 2.10. When a 30-token question sits beside 300 tokens of material, it is 9% of what the model reads. Beside 300,000 tokens, it is 0.01%. The question is still there, but it is a tiny part of what the model's attention is spread over.

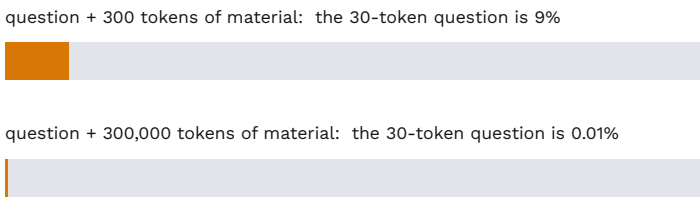


Figure 2.10 How much of the input is the actual question, with a little material and with a lot.

Experiments show the result. As the input grows, answers get worse, even on simple tasks, and even far below the window's limit (Hong et al. 2025). This is called **context rot**.

So a big window does not mean you can stop choosing what to send. Module 3 is about choosing well.

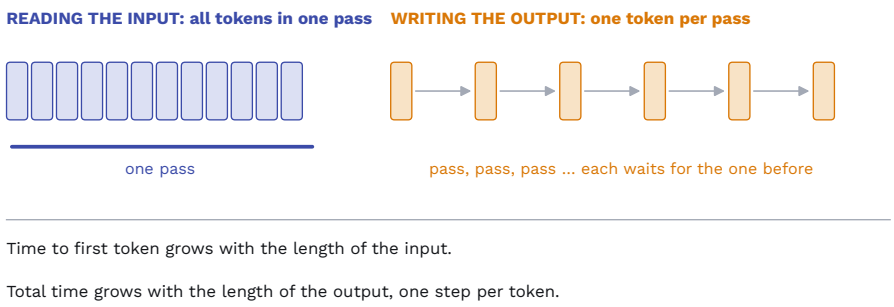


Figure 2.11 The model reads all the input in one go, but writes the output one token at a time.

2.8 Reading is fast, writing is slow

Figure 2.11 shows the difference between how a model handles your input and its own output.

Your input is known from the start, so the model reads all of it in one go. Its output is not. It must write token 1 before it can write token 2, so it works through the output one token at a time. Each output token takes much more computer time than each input token. That is why providers charge around four to five times more for output.

You will notice two speeds:

- **Time to first token**: how long before anything appears. It grows with the length of your input, because all of it is read first.
- **Tokens per second**: how fast the rest arrives once it starts.

Showing the reply word by word as it is written, called **streaming**, makes a slow reply feel fast, because the user starts reading early.

Providers also cut costs behind the scenes. They reuse the work when many requests start with the same text (**prefix caching**; Module 3 shows how to benefit), and they compress the model's numbers to save memory and time (**quantisation**, the "compressed" bar in Figure 2.7).

2.9 Same question, different answer, even at temperature 0

At temperature 0, the model should always pick its top token, so the same question should always get the same answer. Often it does not.

The reason is in how providers run models. Your request is processed together with other people's requests, and the size of that group changes every moment. That

changes the order in which the computer adds up its numbers, which can change the result in the last few digits. Usually this changes nothing. But when two tokens are almost tied, the tiny difference can flip the choice, and the rest of the reply goes another way (He and Thinking Machines Lab 2025).

The lesson is simple: never build anything that needs the model to give exactly the same text twice. Check the meaning of the answer instead. Module 9 shows how.

2.10 Thinking costs tokens

What is 2 + 2? You answer at once. What is 17 × 24? You probably work it out in steps. Models are similar: on problems with several steps, they do better if they write out their working first.

Thinking models do this automatically. Before the answer, they write hidden reasoning, and a setting usually called **effort** controls how much. You pay for that reasoning like any other output. Figure 2.12 shows what that means for a simple job, sorting a message into one of four categories.

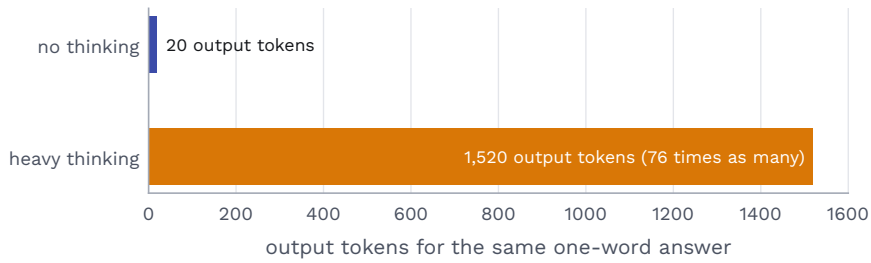


Figure 2.12 Output tokens for sorting one message, with and without heavy thinking. The answer is the same one word.

The answer is the same one word, at 76 times the output cost. For sorting messages, that is waste. For working out a refund across three partial returns and a coupon, the thinking may be what makes the answer right. So compare models by the **cost per finished task**, not by the price per token.

2.11 Big models that still run fast

A bigger model usually gives better answers. But every word passes through the whole model, so a bigger model is also slower and costs more per word.

A **mixture of experts** gets around this. The model is split into many parts, called experts, and for each token a small router sends the work to only a few of them (Figure 2.13). Mixtral 8x7B, for example, has about 47 billion parameters in total, but uses about 13 billion for each token (Jiang et al. 2024).

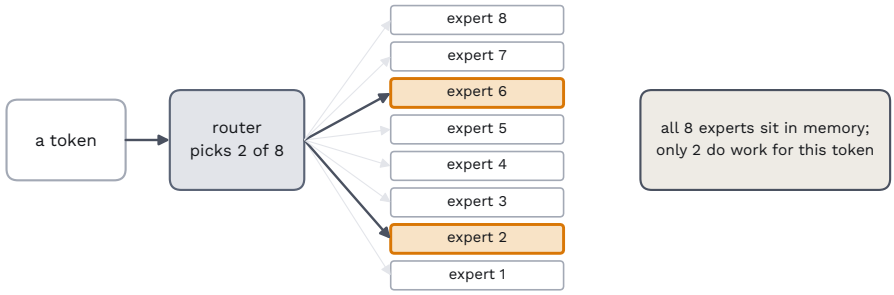


Figure 2.13 A mixture of experts. Each token goes to only two of the eight experts, but all eight must be kept in memory.

The catch: speed depends on the part that is used, but memory depends on the whole. Every expert must be loaded, because the next token might need a different one. Such a model can be as fast as a small model and still need the memory of a large one.

2.12 Images, documents and speech

The same design handles more than text. An image is cut into small squares, and each square is treated like a token, so the model can answer a question about a photo. A scanned page can be read the same way. Speech is cut into short slices of sound, or turned into text first by a separate model.

Images are not free. One image can cost as many tokens as a full page of text, so a screenshot with every question adds up quickly.

2.13 Choosing a model

There is no single best model. There is only the right model for each job.

- **Sorting, extracting and short answers**: a small, fast, cheap model. A large one gives the same answer at many times the cost.
- **Problems with several steps, or writing that must be exact**: a large model, sometimes with thinking turned on.
- **Open-weight models**, which you can download and run yourself: data stays in-house, and they can be cheaper at high volume. But you run the hardware, and "open" does not always mean free for commercial use. Read the licence.
- **Models through an API**: no hardware to manage. Usually the best place to start.

The only reliable way to choose is to test. Try a few models on your own real examples, and compare quality, speed and cost per finished task. Module 9 shows how.

The model knows only what is in front of it at the moment you call it. So what should you put in front of it, and in what order? That is Module 3.

03

Prompts and context

A model knows only what is in the request you send it. Module 2 showed that sending more can make answers worse. This module is about deciding what goes into each request, in what order, and at what cost.

3.1 What goes into one request

In Module 1, a real support request was about 3,000 tokens, and the customer's question was only a small part of it. Figure 3.1 shows the rest.



Figure 3.1 The parts of one support request. Instructions and the tool list are the same every time. The other parts change with every message.

1. **Instructions**, also called the **system prompt**: what the assistant is for, its rules, and the shape of its reply.
2. **The tool list**: the name and description of every function the model may ask your program to run (Module 5).
3. **Earlier messages** in the conversation. The model remembers nothing between calls, so your program sends them again (Module 4).
4. **Policy text** fetched for this question, such as the return rules for earbuds (Modules 6 to 8).
5. **The question** itself: 40 tokens, about 1% of the request.

The model reads all five as one long piece of text. It cannot tell which parts you meant to matter more. Whatever is in the request shapes the answer, and whatever is missing cannot.

Choosing what goes into each request, and in what order, is called **context engineering**. Writing the instructions is **prompt engineering**, which is one part of it (Anthropic Applied AI team 2025). The rest of this module goes through the parts one by one.

3.2 Writing the instructions

Here are instructions many first versions start with.

```
You are a helpful assistant for Tarkika. Answer the customer's question.
```

Suppose you try this on fifty real messages. You find four kinds of failure. It promises refunds that the policy does not allow. When the policy text does not cover a question, it makes up an answer. It replies in English to customers who wrote in Hinglish. And its replies run to two paragraphs.

Each failure gets one instruction:

```
You answer customer questions for Tarkika, an online shop.

Use only the text inside <policy>. If it does not answer the
question, say that a person from the support team will reply.

Never promise a refund or a date. Say what the policy allows,
and that the refund team will confirm it.

Reply in the language the customer used, including Hinglish.
Use at most four short sentences.

Example
Customer: earbuds khol liye, return ho sakta hai?
Reply: Opened earbuds can be returned within 7 days only if they
are faulty. Kya earbuds mein koi problem hai? If yes, our team
will check and confirm the return.
```

Three habits are at work here.

First, every instruction exists because of a failure you saw. Instructions written in advance for problems that never happen cost tokens on every call and give the model more rules to balance. Start short, test on real messages, and add a line only when a test fails (Anthropic Applied AI team 2025).

Second, the policy text is wrapped in `<policy>` markers. The model can then tell your instructions apart from the documents you pass in. This matters more than it looks. A document or an email can contain sentences that read like orders, and Module 12 shows how attackers use that.

Third, one example sets the tone, length and language mix better than a paragraph describing them. But the model copies examples closely. With one example about earbuds, replies may start to sound like that one reply. If you use examples, give two or three that differ from each other.

In LangChain, the instructions go in a `SystemMessage` and the rest follows:

```
from langchain_core.messages import SystemMessage, HumanMessage

INSTRUCTIONS = open("prompts/support.txt").read()   # the fixed text

def ask(question, policy, history):
    messages = [SystemMessage(INSTRUCTIONS),
                *history,                                   # earlier messages
                HumanMessage(f"<policy>\n{policy}\n</policy>\n\n{question}")]
    return llm.invoke(messages)
```

The instructions live in a file of their own, not inside the code. The last section of this module explains why.

3.3 Where you put things changes the answer

Does it matter where in the request the important text sits? Before reading on, guess: start, middle or end?

In a well-known experiment, a model was given 20 documents and a question. Only one document held the answer. The researchers moved that document from first place to last and measured how often the answer was right (Figure 3.2) (Liu et al. 2024).

With the answer in the first document, it was right about 75% of the time. With the answer in the middle, it fell to about 54%. That is worse than the 56% the same model scored with no documents at all. The useful document was there, but hidden among the others, it did more harm than good. Researchers call this **lost in the middle**.

There are two likely reasons. Models learn from text in which the important part usually comes first: titles, headlines, instructions. And they learn to predict the next word mostly from the words just before it, so the end of the input also gets close attention. The middle gets neither.

Newer models do better on this test, but the pattern has not gone away, and it grows as inputs get longer (Hong et al. 2025). So for your requests:

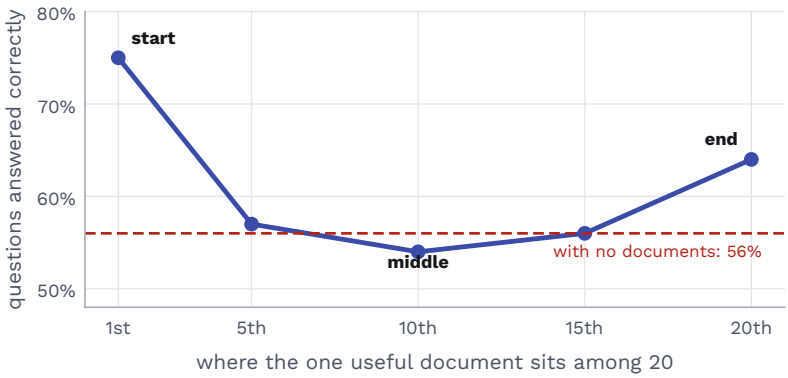


Figure 3.2 How often a model answered correctly, depending on where the one useful document sat among 20. Redrawn approximately from Liu et al. The dashed line is the same model given no documents at all.

- Instructions go at the start.
- The question goes at the end, right after the policy text it needs.
- Inside the policy text, put the most relevant pieces first and last, and the weakest in the middle.

3.4 When the material does not fit

A search over Tarkika's policies, in our example, returns 40 pieces of about 400 tokens each: 16,000 tokens for a budget of 1,200. Sending all of it would cost more, be slower, and, as you just saw, give worse answers. Figure 3.3 shows how to get it down.

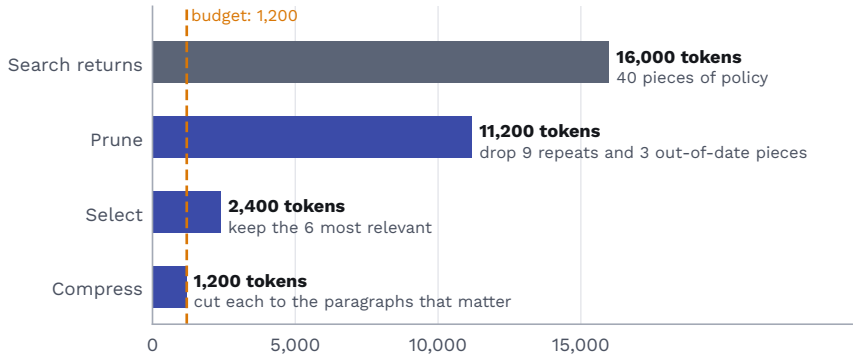


Figure 3.3 Getting 40 search results down to a 1,200-token budget. (Numbers assumed for illustration.)

1. **Prune.** Remove what should never be sent. Here, 9 pieces repeat text found in others, and 3 are from a return policy that changed last year. That leaves 28.

2. **Select.** Keep only the most relevant. A reranker (Module 7) scores each piece against the question, and the top 6 stay. Now 2,400 tokens.
3. **Compress.** Shorten what stays. Each piece is cut to the paragraphs about the product in question, or summarised by a small, cheap model. Now 1,200 tokens.
4. **Order.** Arrange the 6 pieces as in the previous section: best first and last.

There is a fifth operation, which goes the other way. Sometimes a piece says "see section 4.2 for electronics". Without section 4.2 the model cannot answer. **Expanding** means fetching that linked text and adding it.

Each step has a cost. Pruning old versions needs dates on your documents. Selecting needs a reranker, which adds a call and some delay. Compressing with a model adds another call, and a bad summary can drop the one sentence that mattered. So use the cheapest step that brings you within budget: often pruning and selecting are enough.

3.5 A budget for every part

The model in our example accepts 128,000 tokens. The request uses 3,000. Why stay so far below the limit?

Because every token you add is paid for three times. At Module 1's example prices, each extra 1,000 input tokens costs ₹0.35 per message, or ₹42,000 a month at 4,000 messages a day. The reply starts later, because the model reads the whole input before it writes anything. And as Module 2 showed, answers get worse as the input grows.

So decide a budget for each part, and decide in advance what happens when a part goes over it:

Part	Budget	When it goes over
Instructions	700	Remove lines that no test needs
Tool list	600	Send only the tools this request may need (Module 5)
Earlier messages	460	Summarise the older ones (Module 4)
Policy text	1,200	Prune, select, compress
Question	up to 300	For long emails, keep the newest part and drop quoted replies
Reply	300, kept free	Set a maximum length for the output

The last row is easy to forget. The window holds the reply too. A request that fills the window leaves no room for the model to answer.

Write the budget down, and log the actual size of each part on every call. When a cost or a speed problem appears, you can see at once which part grew.

3.6 Prompting a thinking model

A thinking model, as Module 2 showed, writes its own reasoning before it answers. That changes how you instruct it.

Take a harder request. A customer bought three items for ₹1,000, ₹600 and ₹400, with a ₹200 coupon, and returns the second and third. How much is the refund? With an ordinary model, you might write out the steps: list the returned items, find each price, split the coupon, subtract. With a thinking model, those steps get in the way. It plans its own steps, and fixed steps written for one case break on the next case, such as a return of one item out of five.

Instead, describe what a correct answer looks like and how to check it (OpenAI 2025a):

```
Work out the refund for the returned items.
Rules: follow the policy text. Split the coupon across all items
in proportion to their prices.
Before answering, check that each item's refund is no more than
what was paid for it, and that the amounts add up to the total.
Reply with each item's refund and the total.
```

Here, the coupon is split ₹100, ₹60 and ₹40. The refund is ₹540 + ₹360 = ₹900, and the check confirms it. The same prompt works for five items, or two coupons, because it states the goal and the checks, and leaves the path to the model.

The effort setting from Module 2 needs the same care. More effort is not always better: with unclear instructions, a model can spend thousands of tokens going in circles. Start at a medium setting and test higher and lower on your own examples.

3.7 Long tasks: when the context fills up

An agent working on a long task calls a tool, reads the result, calls another, and so on. Every result is added to the context. In Figure 3.4, each step adds about 3,000 tokens. After about 32 steps, a 100,000-token window is full and the run stops.

There are three fixes, and they work best together.

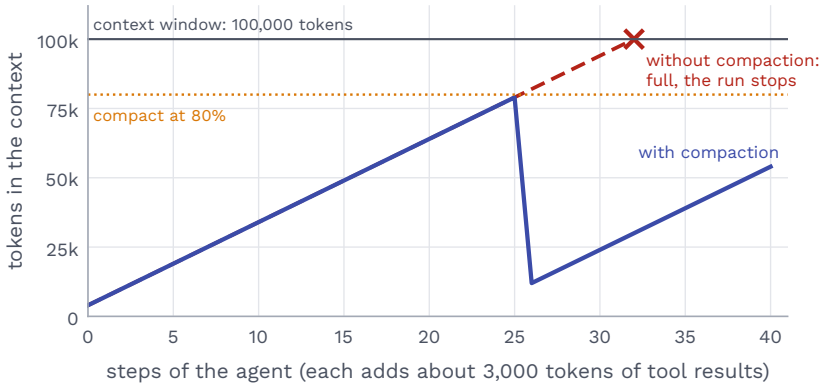


Figure 3.4 Tokens in the context during a long agent run, with and without compaction. (Numbers assumed for illustration.)

1. **Return less from each tool.** An order lookup that returns the full order record, say 2,500 tokens, when the model needs only the status, the date and the amount, about 60 tokens, wastes 97% of what it adds. Design tools to return what the model needs (Module 5).
2. **Fetch just in time.** Instead of loading every order and every policy at the start, give the model short references, such as order numbers and document names, and tools to open them. It fetches the details only when a step needs them (Anthropic Applied AI team 2025).
3. **Compact.** When the context reaches a set level, say 80% of the window, replace the older steps with a short summary of what was found and what is left to do. The run continues in the freed space. The lightest form is to delete old tool results that have already been used.

Compaction has a risk. A summary can drop a detail that turns out to matter twenty steps later. Two habits reduce this. Ask the summary to keep facts, numbers and open questions, even at the cost of length. And let the agent keep a notes file or a to-do list outside the context, which it can read again at any time. Module 14 builds this for long tasks.

3.8 Fixed text first: prompt caching

Look again at Figure 3.1. The instructions and the tool list, 1,300 tokens, are the same in every request. Yet the model reads them again every time, and you pay for them every time.

Providers can save that work. When a new request starts with exactly the same text as a recent one, they reuse the work already done on that shared start, and charge much

less for it. This is **prompt caching**. The reused tokens typically cost about a tenth of the normal price (OpenAI 2025b; Anthropic 2026).

Work it out with Module 1's example prices, assuming cached tokens cost ₹35 per million instead of ₹350:

1. Without caching, 3,000 input tokens cost 3,000 × 350 ÷ 10,00,000 = ₹1.05 per message.
2. With caching, 1,300 cached tokens cost about ₹0.05, and the other 1,700 tokens cost about ₹0.60. Total: about ₹0.64.
3. The saving is about ₹0.41 per message. At 4,000 messages a day, about ₹49,000 a month.

The saving grows with the fixed part. Agents often carry 10,000 tokens of instructions and tools. With 2,000 tokens that change, the input cost falls from ₹4.20 to ₹1.05 a message, a cut of three quarters.

The match must be exact, from the very first token. Figure 3.5 shows the most common mistake.

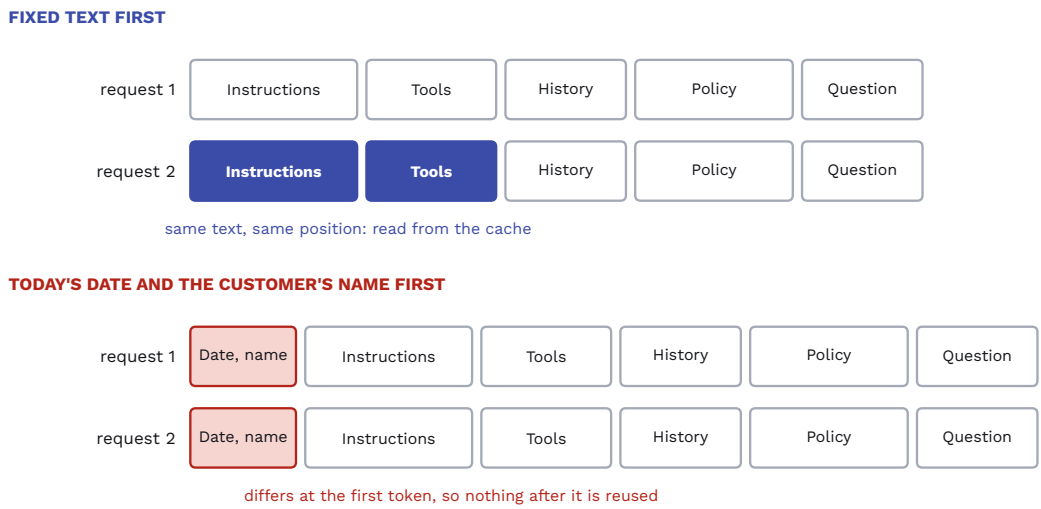


Figure 3.5 Prompt caching reuses only an identical start. Putting anything that changes at the top means nothing after it can be reused.

It is natural to begin the instructions with "Today is 31 October 2026, 2:32 pm. The customer's name is Priya." That one line differs in every request, so the rest of the request never matches anything. The cache is never used, and nothing warns you. Moving the line below the fixed text restores the saving.

So the rule is: fixed text first, changing text last. It agrees with the previous sections. Instructions at the start, the question at the end.

A few details decide whether caching happens at all:

- There is a minimum length, often about 1,000 tokens. A fixed part shorter than that is not cached, again without any error. The exact minimum differs by provider and model.
- The cache expires after a few minutes without use. A busy service keeps it warm, and a quiet one at night pays full price.
- Some providers cache on their own. Others need you to mark where the fixed part ends, and charge a little extra to store it.
- Within one conversation, the earlier messages also stay the same from one turn to the next, so they can be reused too.

Check that it works. In LangChain, `reply.usage_metadata["input_token_details"]` reports the cached tokens as `cache_read`. If it stays at zero on a busy service, something at the start of your request is changing.

3.9 A prompt is code: give it a version

Suppose someone edits one word in the instructions. "You **may** offer to pass the request to the refund team" becomes "You **should** offer…". The change looks harmless and no code changes. But the share of chats passed to the refund team rises from 8% to 15% (assumed numbers), and it takes a week for anyone to connect the rise to the edit.

The instructions decide behaviour just as code does, so treat them as code:

1. Keep each prompt in a file under version control, with a name and a version.
2. Record the version with every call, so any reply can be traced to the prompt that produced it.
3. Before a new version goes live, run it on a fixed set of test messages and compare it with the old one (Module 9).

```
PROMPT_VERSION = "support-v7"
INSTRUCTIONS = open(f"prompts/{PROMPT_VERSION}.txt").read()

reply = llm.invoke(messages,
                   config={"metadata": {"prompt_version": PROMPT_VERSION}})
```

The same applies to everything else that shapes a reply: the model name, the tool descriptions, the examples. A change to any of them is a release (Module 16).

Every request in this module carried earlier messages, and the model forgets them the moment it replies. How your program remembers a conversation, and what it should remember for longer, is Module 4.

References

Anthropic. 2026. *Prompt Caching*. Claude Developer Platform documentation. https://platform.claude.com/docs/en/build-with-claude/prompt-caching.

Anthropic Applied AI team. 2025. *Effective Context Engineering for AI Agents*. Anthropic Engineering blog, 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Cho, Aeree, Grace C. Kim, Alexander Karpekov, et al. 2025. 'Transformer Explainer: Interactive Learning of Text-Generative Models'. *Proceedings of the AAAI Conference on Artificial Intelligence*.

He, Horace, and Thinking Machines Lab. 2025. *Defeating Nondeterminism in LLM Inference*. Thinking Machines Lab: Connectionism.

Hong, Kelly, Anton Troynikov, and Jeff Huber. 2025. *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma technical report. https://research.trychroma.com/context-rot.

Jiang, Albert Q., Alexandre Sablayrolles, Antoine Roux, et al. 2024. *Mixtral of Experts*. arXiv:2401.04088.

Liu, Nelson F., Kevin Lin, John Hewitt, et al. 2024. 'Lost in the Middle: How Language Models Use Long Contexts'. *Transactions of the Association for Computational Linguistics* 12: 157–73.

OpenAI. 2025a. *GPT-5 Prompting Guide*. OpenAI Cookbook, August 2025, and the latest-model guide in the API documentation.

OpenAI. 2025b. *Prompt Caching*. OpenAI API documentation and cookbook 'Prompt Caching 101'.

Petrov, Aleksandar, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. 'Language Model Tokenizers Introduce Unfairness Between Languages'. *Advances in Neural Information Processing Systems (NeurIPS)*.

Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. 'ZeRO: Memory Optimizations Toward Training Trillion Parameter Models'. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*.

Reducing Tokenization Premiums for Low-Resource Languages. 2026. arXiv:2601.13328. https://arxiv.org/abs/2601.13328.

About the author

Sridhar Mukkandi is an applied AI engineer who builds AI agents and the decision systems behind them. He writes for engineers who want AI systems they can trust in production.

sridhar.authorhub@gmail.com